Project - Low Level Design

On

Non Profit Quiz /Tutor Bot

Course Name:

Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AAYUSH BANWADIKAR	EN22ME304002
2	DIVYANSHU MEHTA	EN22EL301024
3	ADITYA KUMAR SAHU	EN22ME304005
4	VINAY PATIDAR	EN22CS3011085
5	YATENDRA VERMA	EN22ME304118

Group Name: 02D10

Project Number: GAI-15

Industry Mentor Name: Mr. Yash Mittal

University Mentor Name: Prof. Divya Kumawat

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue

3. References

1. Introduction

1.1 Scope of the document

The purpose of this document is to describe the design, architecture, and functionality of the Generative AI-based Non-Profit Quiz /Tutor Bot. This document outlines the overall system structure, application workflow, data flow, and technical components used to build the solution.

The system is designed to assist users in learning about the non-profit sector through an interactive quiz-based educational platform powered by Generative Artificial Intelligence and Large Language Models (LLMs). The bot evaluates user responses, provides contextual explanations for incorrect answers, and helps reinforce learning through intelligent feedback.

This document covers the system design, process flow, component architecture, API integrations, and design considerations required to implement the application. It serves as a technical reference for developers, researchers, and stakeholders involved in the development and maintenance of the system.

1.2 Intended Audience

This document is intended for individuals involved in the design, development, deployment, and evaluation of the Non-Profit Quiz Tutor Bot system. The primary audience includes:

- Software Developers and Engineers who will implement and maintain the system architecture.
- AI/ML Engineers responsible for integrating Large Language Models and vector databases for intelligent responses.
- Researchers and Students studying Generative AI applications in education and non-profit domains.
- Non-Profit Organizations interested in adopting AI-based learning tools for training and awareness.

The document provides both technical and conceptual information to help these audiences understand how the system operates and how its components interact.

1.3 System Overview

The Non-Profit Quiz Tutor Bot is an AI-driven educational platform designed to help users improve their knowledge of non-profit sector concepts through an interactive learning experience.

The system presents users with quiz questions derived from donor communications, fundraising strategies, and non-profit operational concepts. After the user submits an answer, the system evaluates the response using a Large Language Model (LLM). Instead of only indicating whether the answer is correct or incorrect, the AI generates detailed explanations and contextual feedback to help users understand the correct concept.

The system also uses a Vector Database to store knowledge related to non-profit topics, allowing the AI model to retrieve relevant information and provide accurate explanations. The chatbot interface enables users to interact with the system naturally, making the learning experience more engaging.

Key technologies used in the system include:

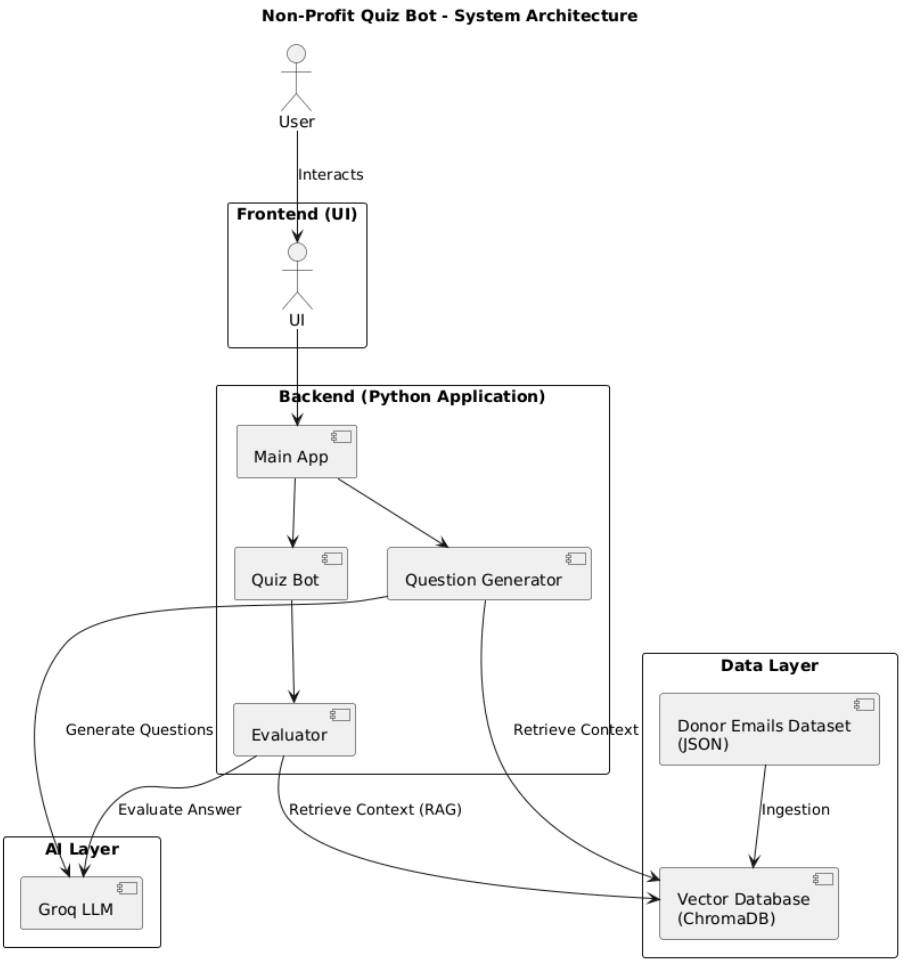
- **Python** for backend development
- **Large Language Models (LLMs)** for intelligent evaluation and explanation
- **Vector Databases (FAISS/ChromaDB)** for semantic search
- **HTML, CSS and JavaScript** for building the chatbot interface
- **API integrations** for connecting the application with AI models

2. System Design

2.1 Application Design

The Non-Profit Quiz/Tutor Bot is designed as a modular AI-driven system that processes donor email data to generate and evaluate learning assessments. The application begins by loading donor emails from a JSON dataset, which are then split into smaller chunks and converted into embeddings for storage in a vector database (ChromaDB). This enables efficient semantic retrieval of relevant information. The question generation module uses a Large Language Model (LLM) via the Groq API to create context-based quiz questions from the stored data. During the quiz, the user interacts with the system through the quiz bot module, which displays questions and collects responses. The evaluator module then analyzes

the user's answers using a combination of direct matching (for objective questions) and LLM-based evaluation (for descriptive answers). It also retrieves relevant email context using a Retrieval-Augmented Generation (RAG) approach to generate accurate and meaningful explanations. The system is controlled by a main controller that manages data ingestion, quiz flow, and evaluation, thereby creating a personalized learning loop that helps users improve their understanding of nonprofit communication practices.



The Non-Profit Quiz Tutor Bot follows a layered application architecture consisting of four main layers: Frontend (User Interface), Backend (Python Application), AI Layer, and Data Layer. Each layer performs a specific function and communicates with other components to deliver an interactive AI-based learning experience. The architecture ensures modularity, scalability, and efficient data processing.

1. User Interface (Frontend Layer)

The Frontend Layer acts as the interaction point between the user and the system. Users access the application through a chatbot-style interface where they can answer quiz questions and receive explanations.

The interface is designed to be simple and interactive so that users can easily engage with the learning system. It sends user responses to the backend for processing and displays the feedback generated by the AI system. Technologies such as Streamlit or web-based UI frameworks can be used to build this layer.

The main responsibilities of the frontend include:

- Accepting user responses to quiz questions
- Displaying quiz questions generated by the system
- Showing evaluation results and AI-generated explanations
- Managing user interaction with the chatbot interface

2. Backend Layer (Python Application)

The Backend Layer is the core processing unit of the application. It manages the logic required for generating quiz questions, evaluating answers, and retrieving contextual knowledge from the database.

The backend consists of several modules:

- **Main Application Module:** The main application module controls the overall workflow of the system. It manages communication between the frontend, quiz engine, evaluation system, AI model, and database.

- **Quiz Bot Module:** The Quiz Bot module manages quiz interactions. It sends quiz questions to the user, collects answers, and forwards them for evaluation.

- **Question Generator:** The evaluator analyzes the user's answers and determines whether they are correct or incorrect. It sends the response to the AI model for deeper analysis and explanation generation.

3. AI Layer

The AI Layer is responsible for intelligent reasoning and explanation generation. This layer integrates a Large Language Model (LLM) **Groq API - LLaMA 3.1** to perform natural language understanding and generate contextual responses.

The AI model performs the following functions:

- Evaluates the correctness of user answers
- Generates explanations for incorrect answers
- Provides contextual feedback to enhance learning
- Uses retrieved knowledge to ensure accurate responses

This layer uses a Retrieval-Augmented Generation (RAG) approach, where the AI model retrieves relevant information from the vector database before generating responses. This improves accuracy and reduces hallucinations.

4. Data Layer

The Data Layer manages the storage and retrieval of knowledge used by the system. It contains structured and processed information related to non-profit sector topics.

The main components of this layer include:

- **Donor Emails Dataset (JSON):** The dataset contains donor communication examples and non-profit related content stored in JSON format. This dataset is used to train and support the quiz generation process.
- **Vector Database (ChromaDB):** The dataset is converted into embeddings and stored in a vector database. This allows the system to perform semantic search and retrieve relevant context when evaluating answers or generating questions.

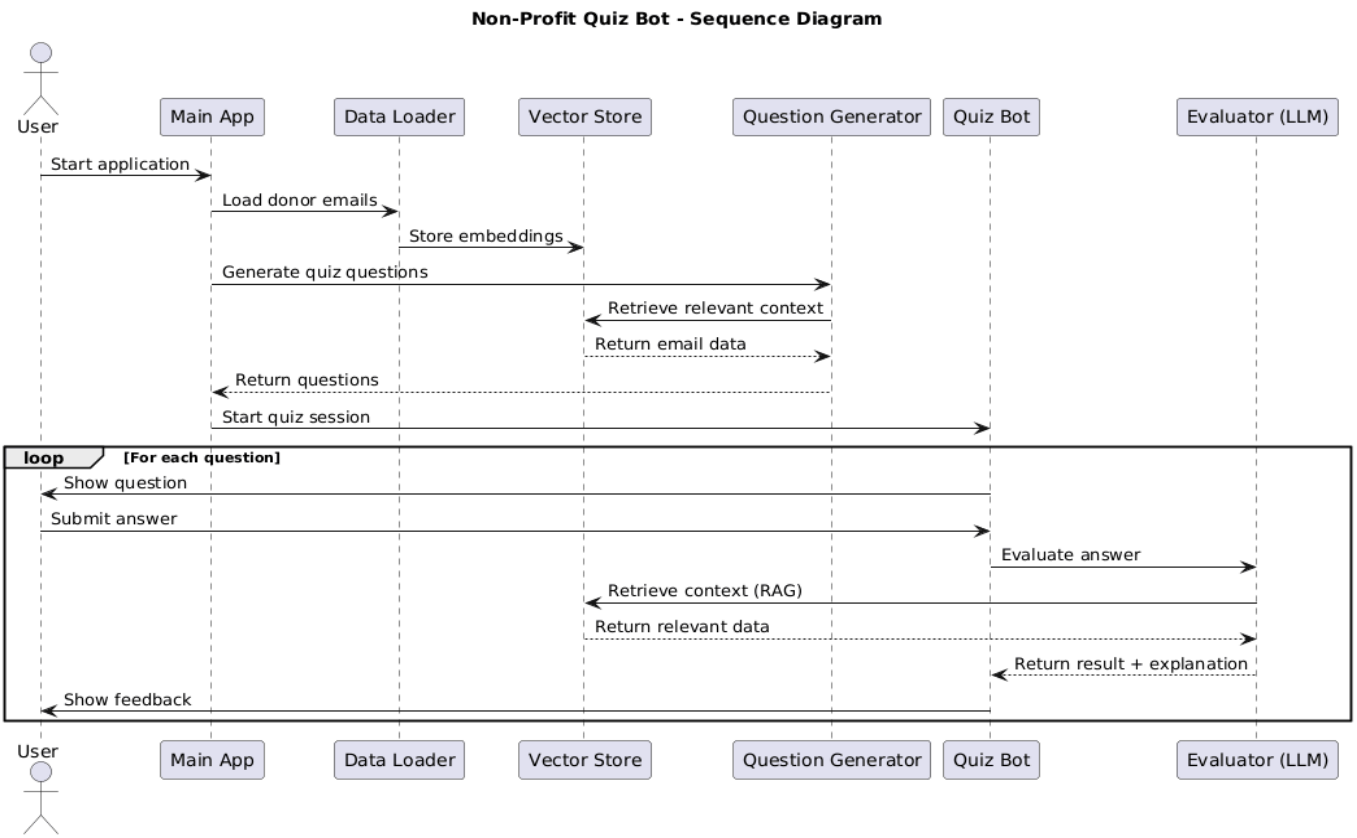
The vector database supports the **Retrieval-Augmented Generation (RAG)** mechanism, enabling the AI model to access accurate contextual information before generating explanations.

5. System Workflow

The interaction between different components follows a structured workflow:

1. The user interacts with the frontend interface and submits an answer.
2. The frontend sends the response to the backend main application.
3. The Quiz Bot module manages the quiz session and forwards the answer to the Evaluator module.
4. The evaluator retrieves relevant context from the Vector Database (ChromaDB) using semantic search.
5. The retrieved information is sent to the Groq LLM in the AI layer.
6. The AI model evaluates the answer and generates an explanation.
7. The result is returned to the backend and displayed to the user interface.

2.2 Process Flow



- User starts the quiz from the interface
- System generates questions using the Question Generator
- Question Generator retrieves relevant data from the Vector Database
- Questions are sent to the user via the Quiz Bot
- User submits an answer
- Quiz Bot forwards the answer to the Evaluator
- Evaluator checks correctness (logic + LLM)
- Relevant context is retrieved from the Vector Database (RAG)
- Context and answer are sent to the LLM (Groq + LLaMA)
- LLM generates explanation/feedback
- Result is returned to Quiz Bot
- Final feedback is displayed to the user

2.3 Information Flow

User Input	The user enters responses through the chatbot interface.
Input Processing	The application processes the input and sends it to the evaluation module.
Knowledge Retrieval	The system queries the vector database to retrieve relevant information.
AI Analysis	The LLM analyzes the response and generates an explanation.
Output Generation	The processed output is sent back to the chatbot interface.
User Feedback	The user receives the result along with educational feedback.

2.4 Components Design

The system consists of several components that work together to deliver the final functionality

.

- **Main Application (main.py):** Acts as the entry point of the system and controls the overall workflow. It manages user modes such as quiz, tutor, and ingestion, and coordinates interaction between all components.

- **Quiz Bot:** Handles user interaction by displaying questions, collecting responses, and showing results. It connects the user interface with backend logic and ensures smooth quiz flow.

- **Question Generator:** Responsible for generating quiz questions using the LLM. It retrieves relevant data from the vector database and creates structured questions of different types like MCQ, True/False, and short answer.

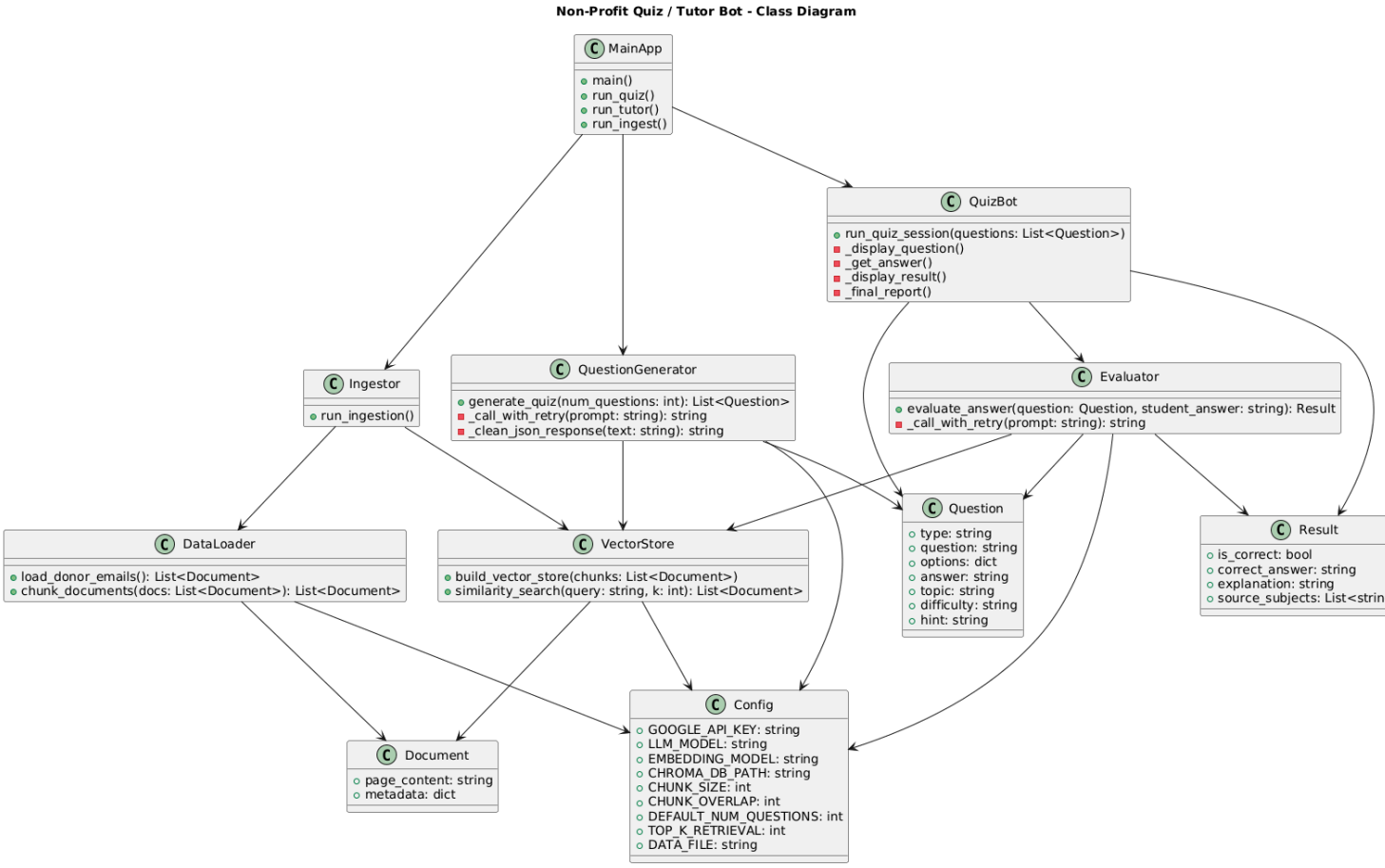
- **Evaluator:** Evaluates user responses using both direct comparison (for objective questions) and LLM-based analysis (for descriptive answers). It also generates contextual explanations using the RAG approach.

- **Data Loader:** Loads donor email data from JSON files and converts it into structured document format for further processing.

- **Vector Store (ChromaDB):** Stores embeddings of donor email content and enables semantic search. It retrieves relevant information for question generation and evaluation, forming the core of the RAG system.

- **Ingestion Module:** Processes raw data by performing chunking and embedding generation, and stores it into the vector database. This step prepares the data for efficient retrieval.

- **LLM (Groq API + LLaMA 3.1):** Performs key AI tasks such as generating questions, evaluating answers, and providing explanations. It works with retrieved context to ensure accurate and meaningful responses.



Class Diagram

2.5 Key Design Considerations

- **Scalability**
  The system architecture allows easy expansion to support more users and larger knowledge bases.
- **Accuracy of AI Responses**
  The use of a vector database ensures that the AI retrieves reliable and relevant knowledge.
- **User Experience**
  The chatbot interface is designed to be simple, interactive, and user-friendly.
- **Modularity**
  Different components are separated so that updates can be implemented without affecting the entire system.

- **Security and Privacy**
  User data and responses are handled securely to ensure privacy and data protection.
- **Performance Efficiency**
  Efficient retrieval mechanisms and optimized AI queries ensure fast response times.

2.6 API Catalogue

API Name	Module	Input	Output	Purpose
load_donor_emails()	Data Loader	JSON file path	List of Documents	Loads donor email dataset
chunk_documents()	Data Loader	List of Documents	Chunked Documents	Splits emails into smaller parts
build_vector_store()	Vector Store	Document chunks	Vector DB created	Stores embeddings in ChromaDB
similarity_search()	Vector Store	Query, k	Relevant Documents	Retrieves related email context
generate_quiz()	Question Generator	Number of questions	List of Questions	Generates quiz using LLM
evaluate_answer()	Evaluator	Question, User Answer	Result (correct/incorrect + explanation)	Evaluates answers using LLM
run_ingestion()	Ingestion	None	Vector DB ready	Executes full data pipeline
run_quiz_session()	Quiz Bot	List of Questions	Score + Feedback	Runs quiz interaction
main()	Main Controller	CLI arguments	System execution	Controls overall application flow

3. References

[1] Groq Inc., *"Groq API Documentation."* [Online]. Available: https://console.groq.com/docs. [Accessed: Mar. 2026].

[2] Meta AI, *"LLaMA 3.1 – Large Language Model."* [Online]. Available: https://ai.meta.com/llama/. [Accessed: Mar. 2026].

[3] LangChain, *"LangChain Documentation."* [Online]. Available: https://python.langchain.com. [Accessed: Mar. 2026].

[4] Chroma, *"ChromaDB Documentation."* [Online]. Available: https://docs.trychroma.com. [Accessed: Mar. 2026].

[5] P. Lewis *et al.*, *"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,"* Facebook AI Research, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401.

[6] N. Reimers and I. Gurevych, *"Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,"* 2019. [Online]. Available: https://www.sbert.net.

[7] Python Software Foundation, *"Python Documentation."* [Online]. Available: https://docs.python.org/3/. [Accessed: Mar. 2026].